

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION with Open CV

COURSE CODE: DAS0204

NAME : V G ABBIRAMY

REG NUMBER : 192524097

COURSE OUTCOME COVERED

CO3: *Generate 3D models from images.* (BTL 4)

Assessment Tool Weightage: 40%

Instructions: Perform the experiment using the laboratory performance rubric Understanding of Concepts, Experimental Design, Use of Tools, Analysis of Results, Documentation.

EXPERIMENT 1 – PREPROCESSING IMPACT ANALYSIS

Aim

To study the effect of preprocessing techniques such as noise removal and normalization on image quality and feature detection.

Objective

To compare the original image with a preprocessed image and analyze how preprocessing improves feature detection accuracy and reliability.

Theory

Image preprocessing is performed before feature extraction to improve the quality of an image. Noise removal eliminates unwanted variations, while normalization improves the intensity distribution.

Common preprocessing techniques include:

- Gaussian filtering

- Median filtering
- Histogram equalization
- Normalization

Noise can produce false features. Therefore, preprocessing helps improve the reliability of subsequent computer vision operations.

Tools Required

- Python
- OpenCV
- NumPy
- Matplotlib
- Sample image

Experimental Procedure

1. Read the input image.
2. Display the original image.
3. Convert the image to grayscale.
4. Add or consider image noise.
5. Apply Gaussian or median filtering.
6. Apply normalization or histogram equalization.
7. Perform feature detection on the original image.
8. Perform feature detection on the preprocessed image.
9. Compare the detected features.
10. Record and analyze the results.

OpenCV Implementation

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
img = cv2.imread("image.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
filtered = cv2.GaussianBlur(gray, (5, 5), 0)

normalized = cv2.normalize(
    filtered, None, 0, 255, cv2.NORM_MINMAX
)

plt.figure(figsize=(10, 6))

plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")

plt.subplot(1, 3, 2)
plt.imshow(filtered, cmap="gray")
plt.title("Filtered")

plt.subplot(1, 3, 3)
plt.imshow(normalized, cmap="gray")
plt.title("Normalized")

plt.show()
```

Result

The preprocessed image contains less noise and improved intensity distribution. Feature detection becomes more reliable because unwanted image variations are reduced.

Analysis

Preprocessing improves image quality and can reduce false feature detections. However, excessive smoothing may remove useful details. Therefore, preprocessing parameters must be selected carefully.

Conclusion

Preprocessing improves the quality of images and provides better input for feature detection algorithms.

EXPERIMENT 2– IMAGE REPRESENTATION STUDY

Aim

To convert an image into grayscale and binary representations and study their effect on edge detection and feature extraction.

Objective

To understand how different image representations affect computer vision processing.

Theory

A color image normally contains three channels: Red, Green and Blue. A grayscale image contains a single intensity channel.

A binary image contains only two intensity values, normally 0 and 255. Binary representation is useful for segmentation and object extraction.

Experimental Procedure

1. Read the input image.
2. Convert it into grayscale.
3. Convert the grayscale image into binary using thresholding.
4. Display all three representations.
5. Apply edge detection.
6. Compare the results.

OpenCV Implementation

```
import cv2  
import matplotlib.pyplot as plt
```

```
img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(
    gray, 127, 255, cv2.THRESH_BINARY
)

edges = cv2.Canny(gray, 50, 150)

plt.figure(figsize=(10, 6))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")

plt.subplot(2, 2, 3)
plt.imshow(binary, cmap="gray")
plt.title("Binary")

plt.subplot(2, 2, 4)
plt.imshow(edges, cmap="gray")
plt.title("Edges")

plt.show()
```

Result

The grayscale image preserves intensity information while the binary image separates foreground and background based on the selected threshold.

Analysis

Grayscale representation is useful for edge detection because it simplifies processing. Binary representation is useful when clear separation between objects and background is required.

Conclusion

Different image representations are selected according to the requirements of the computer vision application.

EXPERIMENT 3 – EDGE DETECTION COMPARISON

Aim

To compare Sobel and Canny edge detection methods on the same image.

Objective

To analyze the effectiveness of Sobel and Canny operators under different image conditions.

Theory

Sobel detects edges by calculating image gradients in horizontal and vertical directions.

Canny is a multi-stage edge detector involving:

1. Noise reduction
2. Gradient calculation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking

Procedure

1. Read the image.
2. Convert it to grayscale.

3. Apply Sobel edge detection.
4. Apply Canny edge detection.
5. Display both outputs.
6. Compare edge quality.

OpenCV Implementation

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel = cv2.magnitude(sobel_x, sobel_y)

canny = cv2.Canny(gray, 50, 150)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel")

plt.subplot(1, 2, 2)
plt.imshow(canny, cmap="gray")
plt.title("Canny")

plt.show()
```

Result

Sobel detects gradients in the image, while Canny produces thinner and more continuous edges.

Analysis

Sobel is simple and useful for gradient analysis. Canny generally produces cleaner edges because it includes noise reduction and threshold-based edge selection.

Conclusion

Canny is generally more suitable when accurate and well-defined edges are required, while Sobel is useful for basic gradient-based analysis.

EXPERIMENT 4 – HARRIS CORNER DETECTION ANALYSIS

Aim

To detect corner points in images containing different textures and patterns using the Harris Corner Detection method.

Objective

To analyze the distribution and significance of detected corners.

Theory

A corner is a point where intensity changes significantly in more than one direction.

Harris Corner Detection analyzes changes in intensity around a pixel and identifies locations that behave like corners.

Procedure

1. Read the image.
2. Convert it into grayscale.
3. Convert it to floating-point format.
4. Apply Harris corner detection.
5. Set the required parameters.
6. Mark detected corners.

7. Display the result.

OpenCV Implementation

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = cv2.float32(gray)

corners = cv2.cornerHarris(
    gray,
    blockSize=2,
    ksize=3,
    k=0.04
)

img[corners > 0.01 * corners.max()] = [0, 0, 255]

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Harris Corners")
plt.show()
```

Result

Corner points are highlighted in regions containing strong changes in intensity.

Analysis

Images with rich textures generally contain more detectable corners. Uniform regions contain fewer corners.

Conclusion

Harris Corner Detection is useful for identifying important points that can later be used for feature matching and image analysis.

EXPERIMENT 6 – HOUGH TRANSFORM FOR SHAPE DETECTION

Aim

To detect lines or circles in an image using the Hough Transform.

Objective

To understand how Hough Transform can identify geometric shapes from edge information.

Theory

The Hough Transform converts image points into a parameter space and identifies shapes based on accumulated votes.

It can be used to detect:

- Lines
- Circles

Procedure

1. Read the image.
2. Convert it to grayscale.
3. Apply Gaussian blur.
4. Apply Canny edge detection.
5. Apply Hough Line Transform.
6. Draw detected lines.
7. Display the result.

OpenCV Implementation

```
import cv2
```

```

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5, 5), 0)

edges = cv2.Canny(blur, 50, 150)

lines = cv2.HoughLinesP(
    edges,
    1,
    3.14159 / 180,
    threshold=50,
    minLength=50,
    maxLineGap=10
)

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(
            img,
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            2
        )

cv2.imshow("Detected Lines", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Result

Straight lines present in the image are detected and displayed.

Analysis

The accuracy of Hough detection depends strongly on threshold and line-length parameters. Low thresholds may produce false detections, while high thresholds may miss weak lines.

Conclusion

Hough Transform is effective for detecting geometric structures such as lines and circles.

EXPERIMENT 7 – FEATURE MATCHING USING SIFT

Aim

To extract and match features between two images using SIFT.

Objective

To study the robustness of SIFT under scale, rotation and illumination changes.

Theory

SIFT stands for **Scale-Invariant Feature Transform**. It detects distinctive keypoints and generates descriptors representing local image regions.

SIFT features are designed to be robust to changes in:

- Scale
- Rotation
- Illumination

Procedure

1. Read two images.
2. Convert both images into grayscale.
3. Create a SIFT detector.

4. Detect keypoints.
5. Generate descriptors.
6. Match descriptors.
7. Filter good matches.
8. Display the matches.

OpenCV Implementation

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread("image1.jpg")
img2 = cv2.imread("image2.jpg")

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher()

matches = bf.knnMatch(des1, des2, k=2)

good = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

result = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
```

```
    good, None,  
    flags=2  
)  
  
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))  
plt.title("SIFT Feature Matching")  
plt.show()
```

Result

Corresponding feature points between the two images are connected using lines.

Analysis

SIFT can identify corresponding points even when the second image is rotated or resized. A higher number of correct matches indicates better feature correspondence.

Conclusion

SIFT provides robust feature matching for images affected by scale and rotation changes.

EXPERIMENT 8 – PCA FOR FEATURE REDUCTION

Aim

To implement Principal Component Analysis for reducing the dimensionality of extracted image features.

Objective

To analyze the effect of PCA on feature dimensions and computational efficiency.

Theory

PCA is a dimensionality reduction technique that transforms correlated features into a smaller number of principal components.

The main steps are:

1. Standardization
2. Covariance analysis
3. Eigenvalue/eigenvector calculation
4. Principal component selection
5. Transformation into reduced feature space

Procedure

1. Collect image feature data.
2. Arrange features into a matrix.
3. Standardize the data.
4. Apply PCA.
5. Select the required number of components.
6. Compare dimensions before and after PCA.
7. Analyze information retention.

OpenCV/Python Implementation

```
import numpy as np
```

```
from sklearn.decomposition import PCA
```

```
features = np.array([  
    [2, 4, 5, 7],  
    [3, 5, 6, 8],  
    [4, 6, 7, 9],  
    [5, 7, 8, 10]  
])
```

```
pca = PCA(n_components=2)
```

```
reduced = pca.fit_transform(features)
```

```
print("Original shape:", features.shape)
```

```
print("Reduced shape:", reduced.shape)
```

```
print("Reduced features:")  
print(reduced)
```

Result

The original four-dimensional feature representation is reduced to two principal components.

Analysis

PCA reduces the number of features while attempting to preserve the most important information. Reduced dimensions require less memory and computational time.

Conclusion

PCA is useful for reducing feature dimensionality and improving computational efficiency.

EXPERIMENT 13 – PREWITT VS SOBEL COMPARISON

Aim

To compare Prewitt and Sobel edge detection operators on the same image.

Objective

To study their differences and analyze edge detection performance.

Theory

Both Prewitt and Sobel are gradient-based edge detection operators.

Prewitt uses convolution kernels to calculate horizontal and vertical gradients.

Sobel uses similar kernels but gives greater weight to the center pixels.

Procedure

1. Read the image.

2. Convert it to grayscale.
3. Define Prewitt kernels.
4. Apply Prewitt filtering.
5. Apply Sobel filtering.
6. Compare the resulting edges.

OpenCV Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg", 0)

prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])

prewitt_y = np.array([
    [-1, -1, -1],
    [0, 0, 0],
    [1, 1, 1]
])

px = cv2.filter2D(img, -1, prewitt_x)
py = cv2.filter2D(img, -1, prewitt_y)

prewitt = cv2.addWeighted(
    px, 0.5, py, 0.5, 0
)

sx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
```

```
sobel = cv2.magnitude(sx, sy)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(rewitt, cmap="gray")
```

```
plt.title("Prewitt")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(sobel, cmap="gray")
```

```
plt.title("Sobel")
```

```
plt.show()
```

Result

Both methods detect image boundaries. Sobel generally provides smoother gradient responses because of its weighted kernel.

Analysis

Prewitt is simple and computationally efficient. Sobel provides better noise suppression because its kernel gives greater importance to central pixels.

Conclusion

Both operators are useful for gradient-based edge detection, but Sobel is generally preferred when smoother and more noise-resistant results are required.

EXPERIMENT 15 – ROTATION IMPACT ON FEATURE MATCHING

Aim

To evaluate the effect of image rotation on feature matching accuracy.

Objective

To study the rotation robustness of feature matching algorithms.

Theory

Rotation invariance means that important image features can still be identified when an image is rotated.

Feature detectors such as SIFT are designed to provide robustness against rotation.

Procedure

1. Read the original image.
2. Rotate the image by different angles.
3. Extract features from the original image.
4. Extract features from each rotated image.
5. Match the features.
6. Count the good matches.
7. Compare matching performance.

OpenCV Implementation

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
sift = cv2.SIFT_create()
```

```
kp1, des1 = sift.detectAndCompute(gray, None)
```

```
angles = [0, 30, 60, 90]
```

for angle in angles:

```
h, w = gray.shape
```

```
center = (w // 2, h // 2)
```

```
matrix = cv2.getRotationMatrix2D(  
    center, angle, 1.0  
)
```

```
rotated = cv2.warpAffine(  
    gray,  
    matrix,  
    (w, h)  
)
```

```
kp2, des2 = sift.detectAndCompute(  
    rotated, None  
)
```

```
matcher = cv2.BFMatcher()
```

```
matches = matcher.knnMatch(  
    des1, des2, k=2  
)
```

```
good = []
```

for m, n in matches:

```
    if m.distance < 0.75 * n.distance:
```

```
        good.append(m)
```

```
print(  
    "Rotation:",
```

```
angle,  
"Good Matches:",  
len(good)  
)
```

Result

The number of good matches can be compared for 0°, 30°, 60° and 90° rotations.

Analysis

A robust feature detector should maintain a significant number of correct matches even after rotation. A decrease in matches indicates reduced matching reliability under that transformation.

Conclusion

Rotation-invariant feature descriptors such as SIFT provide better matching performance when images undergo rotational transformations.

EXPERIMENT 17 – NOISE REMOVAL COMPARISON

Aim

To compare different noise removal techniques and determine their effectiveness in removing noise while preserving image details.

Objective

To study the performance of different filtering techniques.

Theory

Noise removal is an important preprocessing operation.

Common filters include:

- Gaussian filter
- Median filter
- Bilateral filter

Gaussian filtering provides smoothing, while median filtering is particularly useful for impulse or salt-and-pepper noise. Bilateral filtering attempts to preserve edges while smoothing.

Procedure

1. Read the input image.
2. Convert it into grayscale.
3. Add or use a noisy image.
4. Apply Gaussian filtering.
5. Apply median filtering.
6. Apply bilateral filtering.
7. Compare the outputs.
8. Analyze noise removal and detail preservation.

OpenCV Implementation

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("noisy_image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gaussian = cv2.GaussianBlur(
    gray, (5, 5), 0
)

median = cv2.medianBlur(
    gray, 5
)

bilateral = cv2.bilateralFilter(
```

```
        gray, 9, 75, 75
    )

plt.figure(figsize=(10, 7))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Noisy Image")

plt.subplot(2, 2, 2)
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Filter")

plt.subplot(2, 2, 3)
plt.imshow(median, cmap="gray")
plt.title("Median Filter")

plt.subplot(2, 2, 4)
plt.imshow(bilateral, cmap="gray")
plt.title("Bilateral Filter")

plt.show()
```

Result

The filters reduce image noise to different degrees while producing different levels of detail preservation.

Analysis

Gaussian filtering provides smooth results but may blur edges. Median filtering is effective for impulse noise. Bilateral filtering provides smoothing while preserving important edges.

Conclusion

The selection of a noise removal method depends on the type of noise and whether preserving edges is important.

Code of Conduct:

I V G ABBIRAMY - 192524097 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G ABBIRAMY

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
----------------------	-----------	--	---	---	---